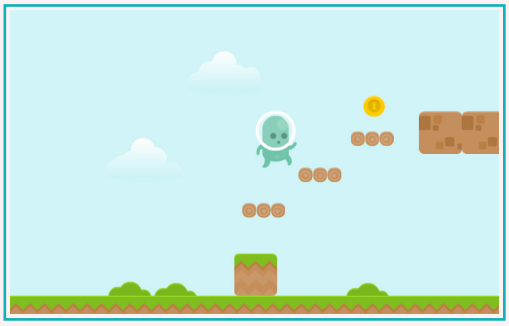
Platformer Game

11

In the last lesson, we learnt about platformer games, sprites, and how to create a sprite. Recall that a platformer character can move on a platform in either direction and can jump when instructed. We added a tiled sprite with a platform behaviour, which keeps our character from falling off.

But a platform game requires much more than this. The game should continue to move forward, and the sprite should be able to jump. In this lesson, we will learn to add 3 new functionalities:

1. Character animation
2. Jump through platform
3. Camera follow



11. Platformer Game

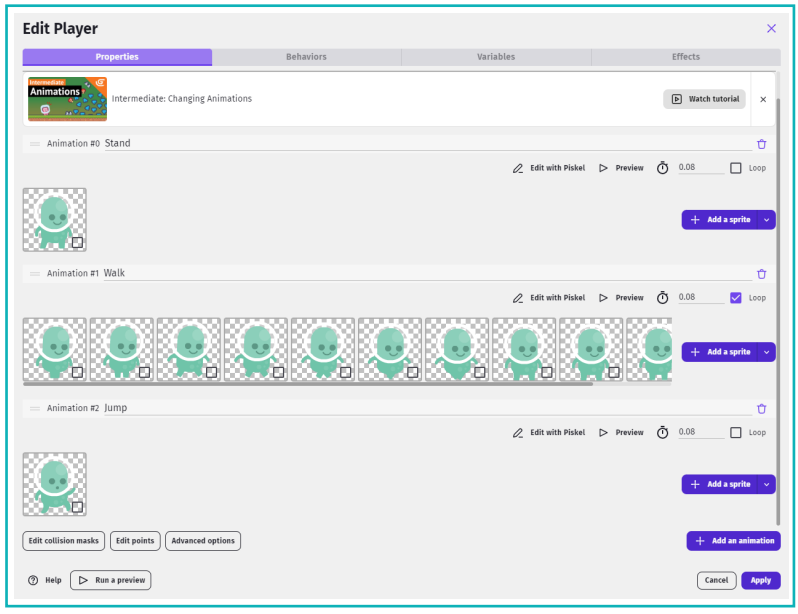
In the previous lesson we explored how behaviors can be used to make a platformer character move. Although the character is moving, the sprite image that we used is in the same position with no animation. When we jump, walk, or talk, our actions change. A game sprite needs to react in the same way.

Animation

To show a walk or run action when a character is moving, we will use “Animations”. An animation allows us to add an image or a series of images to the sprite object.

Sometimes, an object requires more than one animation. This feature allows us to easily separate different animations for diffrent actions.

Scan the QR code to download the assets.



Later, we can use events to switch between animations.

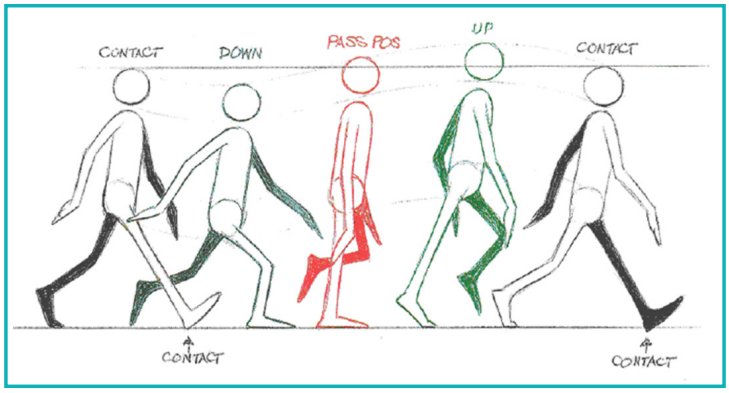
Note:

In objects with multiple animations, it can be difficult to differentiate between animations. It is a good practice to name the animations when there are multiple. If we do not add names, we need to use animation numbers to refer to them.

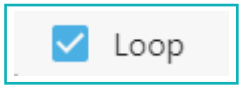
During activities like walking, talking, or running we repeat one action continuously. For example, while talking, we open and close our lips multiple times. Similarly, when we create an activity-based animation for a sprite, we need to repeat animated actions.

Repeating the animation

In animation, a walk cycle is a series of frames or illustrations drawn in a sequence that loops to create an animation of a walking character. The walk cycle is looped over and over. This avoids the need to animate each step separately.



By default, every animation plays only once, which means that it stops as soon as its last frame finishes playing. To repeat an animation, we "loop" the animation by clicking the Loop icon.



In this case, we have looped the "walk" animation.

Let us better understand repeat animations using a clock as an example.

Clocks have numbers 1 to 12 printed at equally spaced intervals around the edge of the face which indicate the hour. 12 sits at the top and on many models, sixty dots or lines are evenly spaced in a ring around the outside of the dial, indicating minutes and seconds. We read time by observing the placement of several "hands", which originate from the centre of the dial:

a. A short, thick "hour" hand.

b. A long, thinner "minute" hand.

c. An even thinner "second" hand.

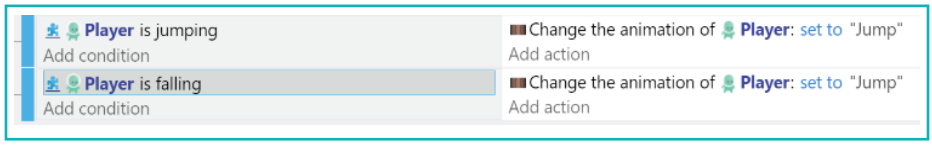
All the hands rotate on repeat around the dial in the direction of increasing numbers. This is called a clockwise direction. For each rotation of the second hand, the minute hand will move from one minute-mark to the next. For each rotation of the minute hand, the hour hand will move from one hour-mark to the next.

All hands of the clock continuously loop this rotating movement at a varying pace.

Using multiple animations with events

After we create multiple animations with their own unique set of images, we can use events to switch between the animations.

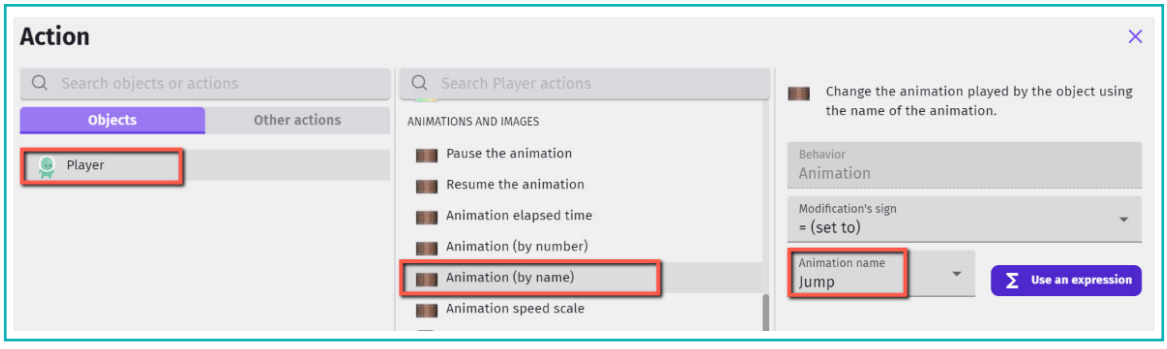
When we set multiple animations for an object, we use the events tab to “Change the animation (by name).” This is controlled in the “Add action” section of the condition. It allows us to switch to the correct animation whenever a condition is met.



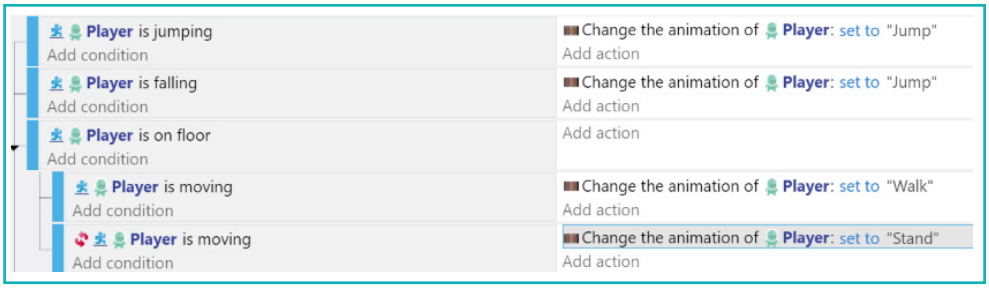
We will use different names when an object has multiple animations to easily differentiate between them.

To add an action to change the animation by its name, choose the ‘Animation (by name)’ condition.

Select the drop-down arrow and choose the animation name as shown.

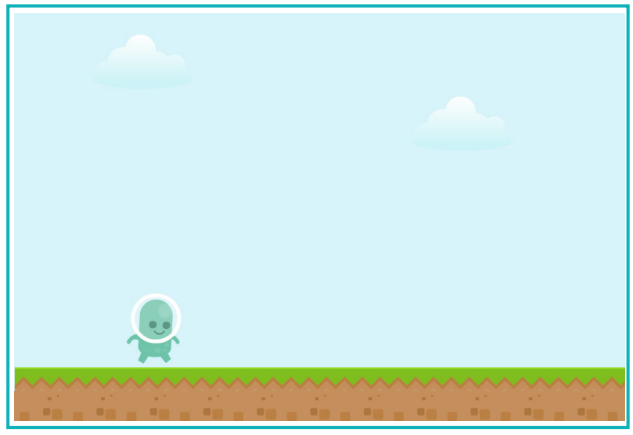


For our platformer, we will use animations with the following events and actions:



They have the following directional conditions:

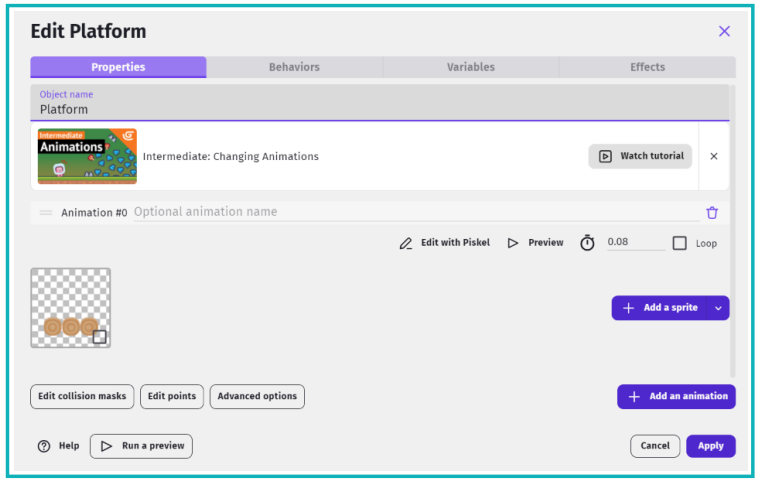
Test and observe the result. We see that the respective animations are played for each movement that we have used as a condition. This includes walk, jump, and flip.



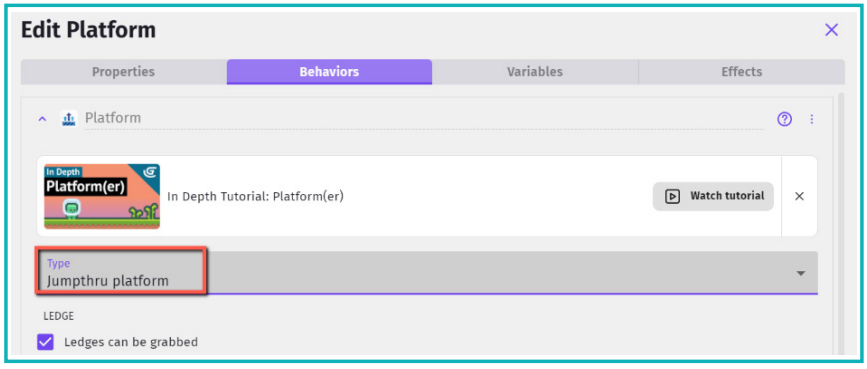
Jumpthru platform

Our game lacks the platform through which the game will proceed. A platformer game could have different types of platforms. One type is a Jumpthru platform. A Jumpthru allows the player to jump onto the platform from beneath it. In GDevelop, we use behaviors to achieve this.

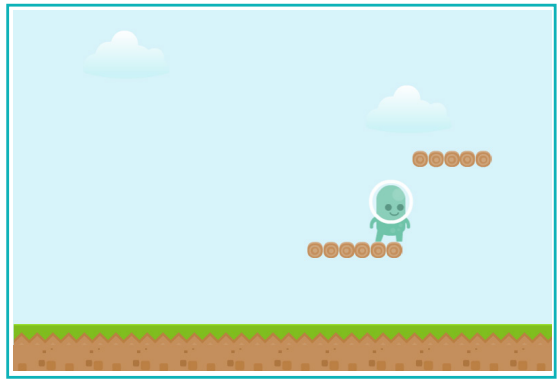
Create a new object using Sprite type.



Add Platform behavior and set type to Jumpthru platform.

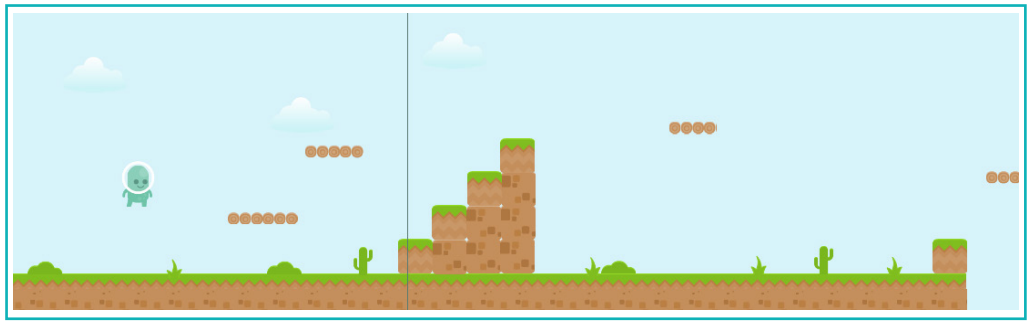


Place some instances of the object onto the scene and test the game. Design rest of the level on your own using various platforms sprite and adding the relevant behaviours. Use your creativity to make it interesting.



Following an object

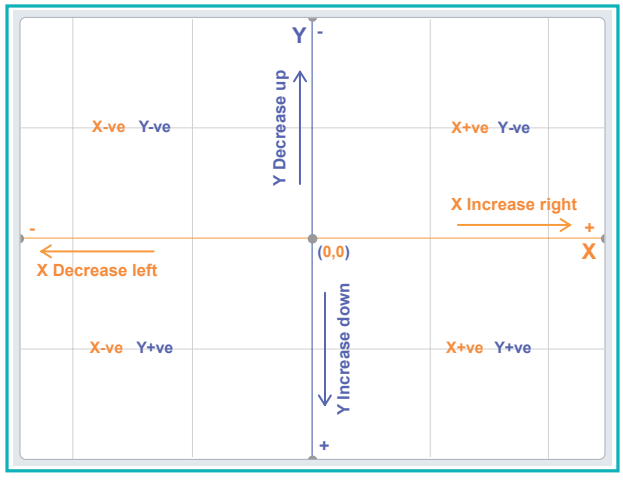
Usually, a platformer game level is bigger than what appears in the player window. Only a small portion of the scene is visible while playing. As the player progresses, other areas become visible. We achieve this by making the camera follow the player object.



Although the entire level has already been designed, only the defined scene area is visible to the player while playing. If the player moves beyond this area, the character stops being visible. To ensure the player is always visible, we will add an action to make the camera follow the player object.

There are several ways to make the camera follow the player. To follow an object, we need to be aware of the object's location. All objects have coordinates. These coordinates correspond to the horizontal position (X-axis) and the vertical position (Y-axis) on the Cartesian plane.

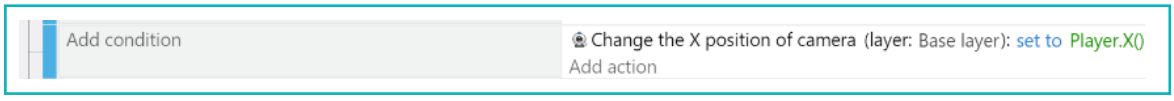
In GDevelop, the X-coordinate number decreases towards the left and increases towards the right. The Y-coordinate number increases while going downwards and decreases while going upwards.



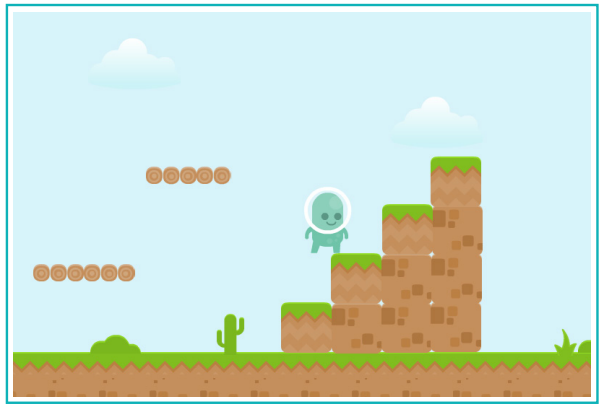
11. Platformer Game

Right now, the action that we have added runs once in each frame. Therefore, if the game runs at 60 frames per second, the action also runs 60 times per second. To correct this, we need to specify a condition.

Since our level is designed horizontally, we will change the camera's x position with respect to the player's x position.



Test the game and check whether it runs properly.



Exercise

Answer in one line

1. What is a Jumpthru platform?

_____

_____

2. Which feature allows us to repeat an animation?

_____

_____

3. How do we determine the location of an object?

_____

_____

11. Platformer Game

Tech Flash

- **Execute every frame** : When a condition is not specified, an action runs once in every single frame.
- **Jumpthru platform** : A Jumpthru platform is a type of platform behaviour that allows the player to jump onto the platform from beneath it.
- **Animation name** : Animation names are used to switch between multiple animations using events.